Assignment 1

Instructions

Enterprise Architecture Design Assignment 1

Marks: 25%

Due date: upload to Brightspace by Friday 3rd April @ 22.00

Word limit: Project report max 1500 words (excluding diagrams/appendices)

Submission format: Brightspace upload (zipped folder) + PDF report + screencast (link to student OneDrive shared video)

Note: *This is an individual assignment, all work must be your own. The University Plagiarism rules apply.*

Overview

You are hired by a small e-commerce company. They are migrating from a single-host Docker Compose setup to Kubernetes because they need:

- safer day-2 operations (rollouts, recovery)
- scaling for bursty traffic (including scale-to-zero where appropriate)
- better diagnosis when partial failures occur (request correlation + clear evidence)

Your task is to design, implement, test, and demonstrate a nanoservices-based checkout workflow on Kubernetes (K3s). Your solution must include KEDA-based scaling and a repeatable troubleshooting/diagnosis workflow using logs, events, and service routing evidence.

You may base your application on the Lab examples (gateway → checkout → pricing + inventory) or extend it. You must keep the system microservices-based (minimum 3 services, plus gateway/entrypoint).

You will be marked on evidence and operational clarity, not just “it works once”.

Functional requirements (baseline)

You must implement a checkout API exposed via HTTP through Kubernetes Ingress (Traefik on K3s). The system must include:

1. Microservices composition

- A public endpoint: POST /api/checkout
- A gateway/edge component that serves:
    - UI at /
    - GET /api/arch returning an architecture label string
    - GET /api/ping for timing/health sanity checks
- A checkout service that composes at least two dependencies over HTTP (e.g., pricing and inventory).
- Dependencies must have health endpoints (/health) and meaningful error handling.

2. *Reliability behavior and partial failures*
    - If a dependency is unavailable, checkout must fail quickly and clearly (no indefinite hangs).
    - You must implement timeouts in the checkout service for dependency calls.
    - You must show at least one partial-failure scenario where the gateway remains up but checkout fails, and explain why.
3. *Scaling with KEDA (required)*
    - At least one user-facing service must be configured for scale-to-zero (min replicas 0) using KEDA.
    - Your system must demonstrate scale-from-zero when traffic arrives and scale-to-zero when idle.
    - You must measure cold vs warm latency for the scaled-to-zero path and present results.
4. *Request correlation / tracing-lite (required)*
    - Support an incoming request header X-Request-Id.
    - That same Request ID must be propagated across the gateway → checkout → dependencies.
    - Logs must include the request ID so you can trace one request across at least two services.
5. *Troubleshooting evidence (required)*
    - You must include a repeatable diagnosis workflow showing:
        - “what exists” (deploy/pods/svc/ingress)
        - failure mode evidence (events, describe output)
        - logs evidence

    - service routing evidence (endpoints/endpointslices)
    - inside-cluster connectivity check (toolbox/curl pod)

R6. Persistence (required)

- Include Postgres deployed to Kubernetes with:
    - Secret-managed credentials
    - PVC-backed storage
- Demonstrate persistence: insert a row, restart the pod, verify the row still exists.
- You may store checkout audit events, request logs, or simple order records—your choice.

Non-functional requirements (baseline)

1. Kubernetes-first

- Your final solution must run on Kubernetes (K3s).
- Compose may be used for local dev only, but is not the final deployment target.

2. Declarative manifests

- Use YAML manifests (no Helm required).
- Your repository must include the manifests to deploy the system from scratch.

3. Security hygiene

- Do not hardcode database passwords in plaintext in manifests.
- Use Kubernetes Secrets for sensitive values.
- Containers should run as non-root where feasible (document exceptions if any).

4. Observability basics

- Each service must provide a /health endpoint.
- Each service must produce logs that are useful for diagnosis (at minimum, request method/path and request ID).

Architecture: required deliverable diagrams

Include at least these two diagrams in Part 1 and in the report.

Diagram A: System architecture (components + routing)

Example (adapt to your design):

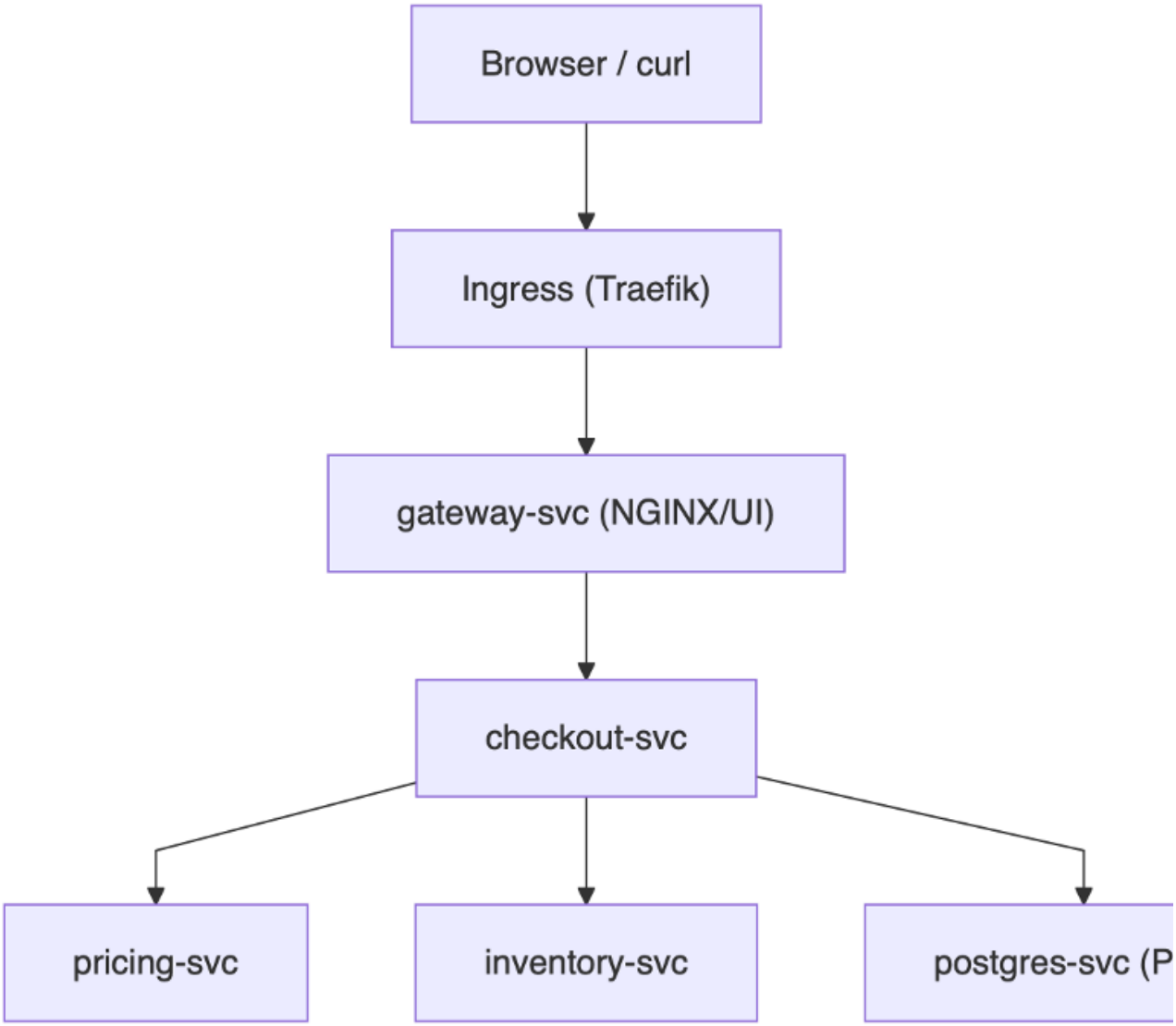
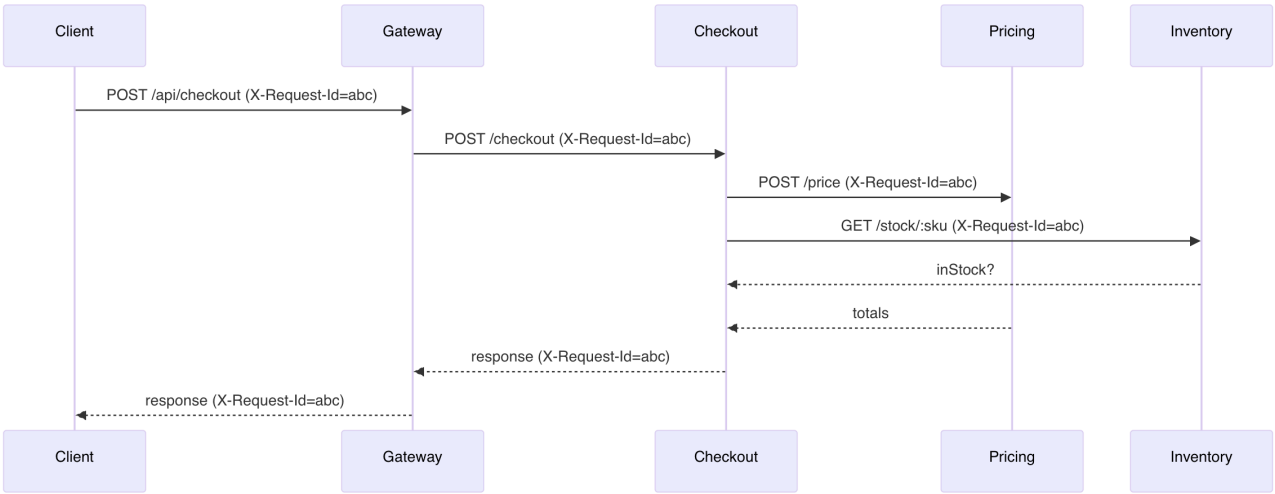


Diagram B: Request flow with correlation ID

Example (adapt to your design):



Deliverables and marking (100 marks)

Part 1 (20%): Solution Requirements and Design

Deliverable: design document with diagrams (include in final report).

- Clear restatement of requirements and scope (what you will build, what you won't).
- Architecture diagrams (components + request flow) accurately match your intended implementation.
- Design rationale: scaling decisions (what scales to zero and why), timeout strategy, failure handling, persistence purpose.

Part 2 (30%): Implementation

Deliverable: repository with source + manifests to deploy.

- Application implementation: microservices composition, correct routes, correct error handling/timeouts.
- Kubernetes manifests: Deployments, Services, Ingress, ConfigMaps/Secrets, probes; clean labels/selectors.
- KEDA implementation: correct scale-to-zero behavior; configuration documented; reproducible install steps.

Part 3 (30%): Testing

Deliverable: Testing report including captured outputs (screenshots or copied command outputs) + any scripts.

- Functional tests: happy path + edge cases (out-of-stock, invalid input).
- Reliability tests: dependency down, bad rollout (bad image tag), readiness failure; shows diagnosis outputs.
- Scaling + performance tests: cold vs warm measurements; scale-from-zero/scale-to-zero evidence; tail latency discussion (basic).

Part 4 (20%): Project report (max 1500 words) + screencast

Deliverables:

- Overall report (≤1500 words)
- Screencast (max 10 minutes) with voice-over to ***clearly*** demonstrate and describe your solution.
- All sources must be fully referenced.

Submissions

No submissions yet. Drag and drop to upload your assignment below.

Drop files here, or click below!

Upload Record Choose Existing

You can upload files up to a maximum of 2 GB.

Comments

Paragraph

Reflect in ePortfolio

Activity Details

Task: Submit to complete this assignment

Due 3 April at 10:00 PM

Last Visited 09 March 2026 2:34 PM